

Informe Técnico

Diego Burgos
Sergio Boada
Geraldine Suarez
Danna Poveda
Ingeniería Mecatrónica
Universidad Militar Nueva Granada

20 de agosto de 2026

1. Descripción del sistema

El sistema implementado está compuesto por:

- Motor DC utilizado como actuador.
- Driver TB6612FNG como etapa de potencia.
- ESP32 como unidad de procesamiento.
- Encoder incremental para medición de velocidad.
- Algoritmo de control discreto ejecutado en tiempo real.

La arquitectura general del sistema se muestra en la Figura.

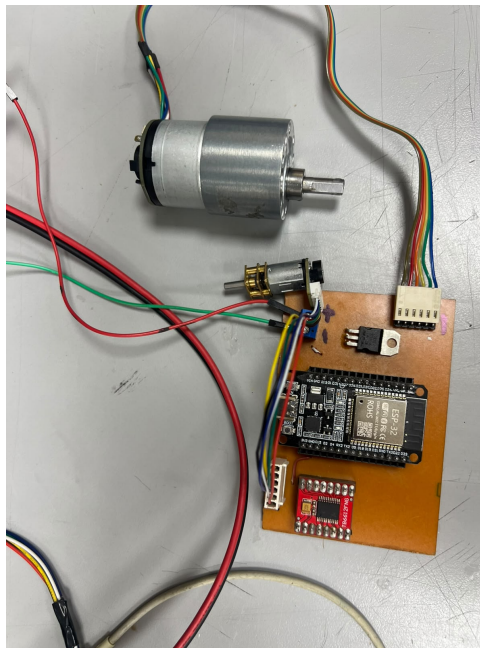


Figura 1: Sistema físico implementado para control del actuador.

2. Adquisición experimental de datos

Para obtener un modelo matemático del actuador se realizó una prueba experimental aplicando diferentes valores de PWM al motor y midiendo la respuesta mediante el encoder.

La adquisición fue realizada utilizando una ESP32, configurando una comunicación serial a una velocidad de 115200 baudios.

El periodo de muestreo utilizado fue:

$$T_s = 0,02s \quad (1)$$

equivalente a:

$$f_s = 50Hz \quad (2)$$

Los datos fueron almacenados mediante un programa desarrollado en Python y guardados en formato CSV para posteriormente realizar el proceso de identificación.

3. Identificación del modelo del actuador

La identificación del sistema fue realizada utilizando la herramienta System Identification de MATLAB.

El comportamiento dominante del motor fue aproximado mediante un modelo de primer orden:

$$G(s) = K_p \frac{1}{1 + T_p s} \quad (3)$$

Los parámetros obtenidos fueron:

$$K_p = 0,046814 \quad (4)$$

$$T_p = 0,002554s \quad (5)$$

Por lo tanto, el modelo identificado corresponde a:

$$(6) \quad G(s) = 0.046814 \frac{1}{1+0.002554s}$$

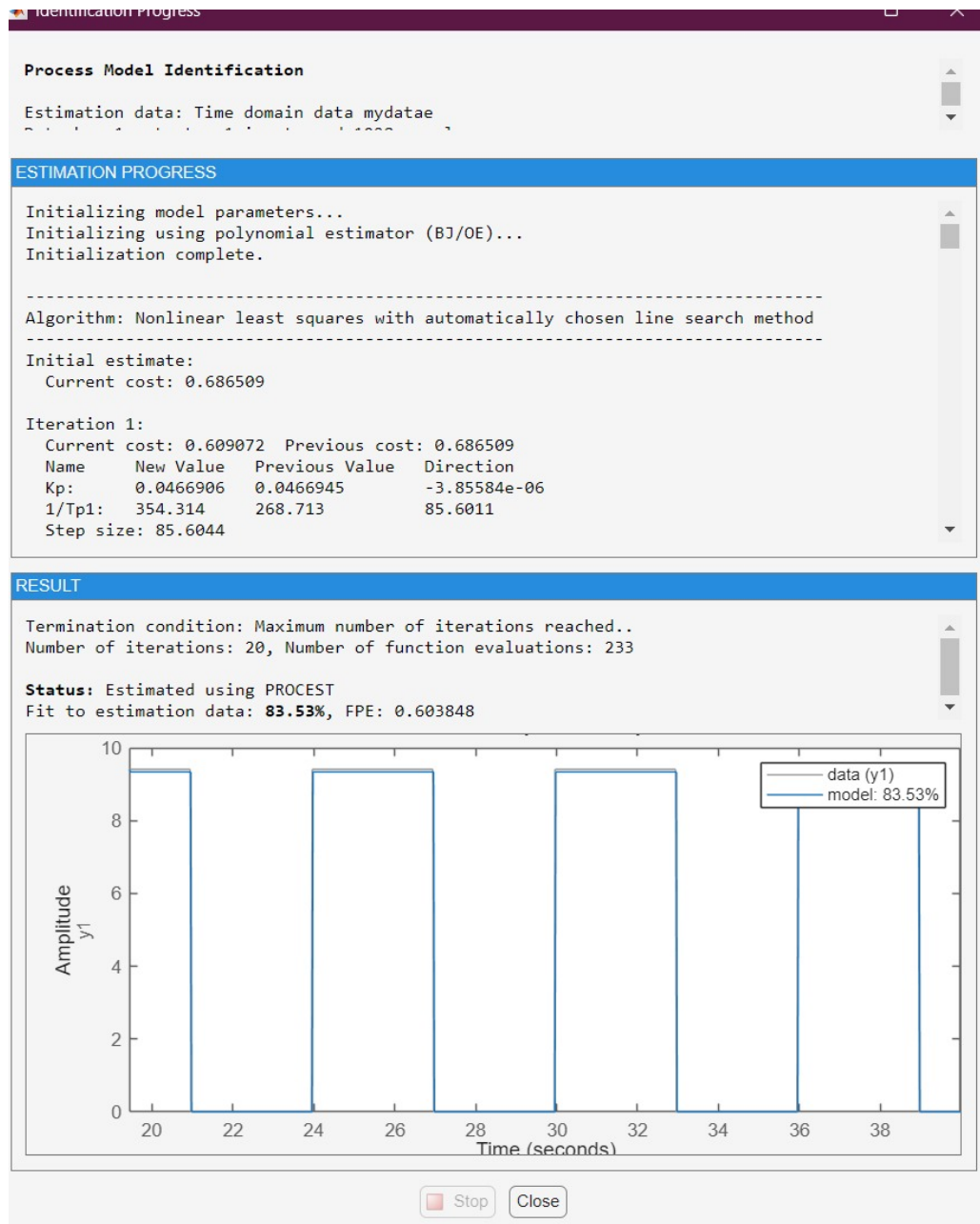


Figura 2: Proceso de identificación del modelo mediante MATLAB.

El modelo obtenido presentó un ajuste del 83.53%, indicando que la dinámica principal del actuador puede representarse adecuadamente mediante un sistema de primer orden.

$$G(s) = \frac{K_p}{1 + T_{p1}s}$$

$$K_p = 0.046814$$

$$T_{p1} = 0.0025643$$

Figura 3: Modelo identificado del actuador.

4. Criterios de diseño

Para el diseño del controlador se establecieron los siguientes criterios de desempeño:

- Tiempo de respuesta reducido.
- Bajo sobreimpulso para evitar movimientos bruscos.
- Error estacionario cercano a cero.
- Seguimiento adecuado de la referencia.
- Estabilidad ante perturbaciones.

Los parámetros seleccionados para el diseño fueron:

Tabla 1: Parámetros de diseño			
Parámetro	Valor	Unidad	
Coficiente amortiguamiento ζ	0.8	-	
Tiempo establecimiento	0.02	s	
Periodo muestreo	0.02	s	
Frecuencia muestreo	50	Hz	

5. Parámetros de diseño según la tarea específica

El diseño del controlador fue realizado considerando la aplicación del actuador dentro de una articulación robótica. A diferencia de una aplicación industrial convencional, donde pueden priorizarse altas velocidades de operación, en sistemas robóticos se requiere un equilibrio entre rapidez, precisión y suavidad del movimiento.

Por esta razón, los parámetros de diseño fueron seleccionados buscando una respuesta suficientemente rápida para ejecutar trayectorias de la articulación, evitando sobreoscilaciones que puedan generar movimientos bruscos o esfuerzos mecánicos innecesarios.

El periodo de muestreo seleccionado de $T_s = 20ms$ permite realizar la adquisición del encoder y ejecutar el controlador en tiempo real sobre la ESP32, garantizando una actualización adecuada de la señal PWM aplicada al motor.

Los principales parámetros considerados fueron:

Tabla 2: Parámetros asociados a la tarea robótica		
Parámetro	Valor	Justificación
Tiempo de muestreo	20 ms	Control en tiempo real
Referencia velocidad	100 RPM	Movimiento controlado del actuador
Resolución encoder	CPR	Medición de velocidad
PWM máximo	255	Limitación del actuador

6. Diseño del controlador discreto

Debido a que la implementación final fue realizada sobre una ESP32, se diseñó un controlador en tiempo discreto.

El modelo continuo fue discretizado mediante un retenedor de orden cero (ZOH).

La estructura del controlador obtenido mediante LGR es:

$$(7) \quad C(z) = K_c \frac{1 - \beta z^{-1}}{1 - z^{-1}}$$

Los parámetros calculados fueron:

Tabla 3: Parámetros del controlador LGR

Parámetro	Valor
K_c	6.9327
β	0.0435
T_s	0.02s

La ecuación implementada en la ESP32 corresponde a:

$$(8) \quad u[k] = u[k-1] + K_c(e[k] - \beta e[k-1])$$

7. Justificación de la técnica de control

Para el control del actuador se seleccionó una estrategia basada en el Lugar Geométrico de las Raíces (LGR) con implementación discreta.

Esta técnica fue seleccionada debido a que permite modificar la ubicación de los polos del sistema y definir directamente características de respuesta como rapidez y amortiguamiento.

Además, al realizar la implementación sobre una ESP32, fue necesario emplear un controlador discreto que pudiera ejecutarse mediante una ecuación de diferencias. El controlador obtenido presenta una estructura simple, reduciendo el costo computacional y permitiendo una ejecución en tiempo real.

La elección de un controlador de velocidad en lugar de un controlador directo de posición se debe a que el motor trabaja con menores esfuerzos cuando se regula la dinámica interna del actuador. Al controlar la velocidad se obtiene indirectamente un comportamiento adecuado de la posición mediante la integración del movimiento.

Adicionalmente, esta implementación facilita futuras ampliaciones del sistema robótico, como la incorporación de modelos cinemáticos, planificación de trayectorias y control basado en cinemática inversa, donde los lazos de velocidad internos son ampliamente utilizados como nivel inferior de control.

8. Implementación del controlador

El controlador fue implementado utilizando el microcontrolador ESP32, encargado de ejecutar el algoritmo cada 20 ms.

El cálculo de velocidad se realiza mediante la diferencia entre muestras consecutivas del encoder:

$$\text{RPM} = \Delta \text{pulsos} \frac{60}{CPR \cdot T_s}$$

(9)

```

// =====
//                               CONFIGURACION PWM
// =====

const int FREQ = 20000;
const int RES = 8;

// Canales PWM para Arduino ESP32 Core 2.x
const int CANAL_A = 0;
const int CANAL_B = 1;

// =====
//                               CONTROLADOR LGR
// =====

// IMPORTANTE:
// Reemplazar estos valores por los calculados en MATLAB

const float Kc = 6.9327;
const float beta = 0.0435;

// =====
//                               TIEMPO DE MUESTREO
// =====

// 20 ms
const float Ts = 0.02;

// =====
//                               ENCODER
// =====

// IMPORTANTE:
// Coloca aquí el número REAL de pulsos por vuelta
// de tu encoder.
//
// Este valor es provisional.
const float CPR = 1320.0;

// =====
//                               REFERENCIA
// =====

// Velocidad deseada del motor en RPM
float referenciaRPM = 100.0;

```

Figura 4: Implementación del controlador en ESP32.

9. Simulación del sistema

Antes de la implementación física se realizó la simulación del sistema en Simulink. La estructura implementada incluye:

- Referencia de velocidad.
- Controlador discreto.
- Planta identificada.
- Realimentación mediante encoder.

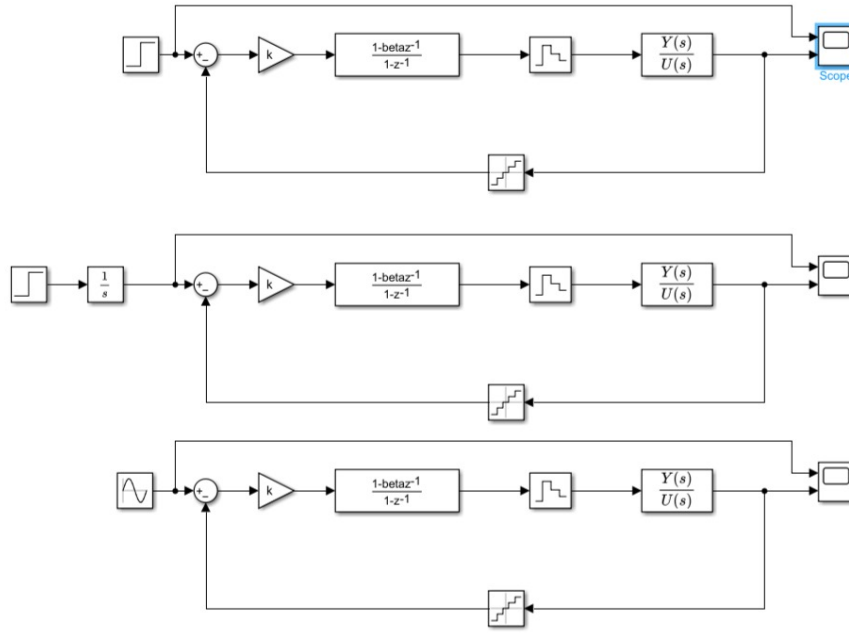


Figura 5: Diagrama de simulación del controlador.

La respuesta obtenida se muestra en la Figura:

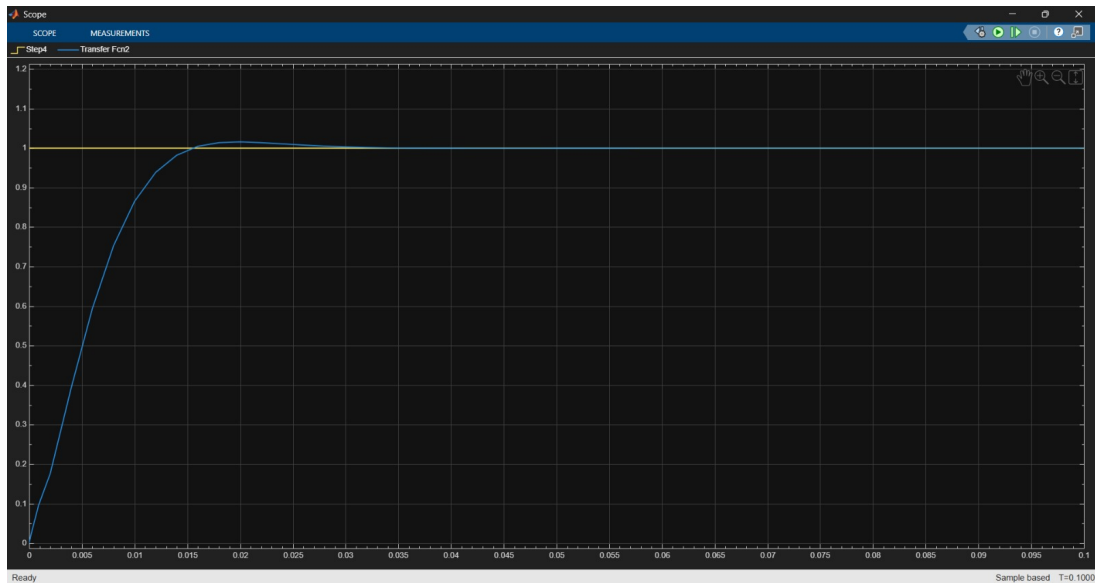


Figura 6: Respuesta del sistema en simulación.

10. Validación experimental

Finalmente se realizaron pruebas sobre el sistema físico.

Los datos obtenidos mediante el encoder fueron comparados con la respuesta simulada.

	A
1	Tiempo_s,PWM,Voltaje_V
2	0.0,200.0,9.41
3	0.02,200.0,9.41
4	0.04,200.0,9.41
5	0.06,200.0,9.41
6	0.08,200.0,9.41
7	0.1,200.0,9.41
8	0.12,200.0,9.41
9	0.14,200.0,9.41
10	0.16,200.0,9.41
11	0.18,200.0,9.41
12	0.2,200.0,9.41
13	0.22,200.0,9.41
14	0.24,200.0,9.41
15	0.26,200.0,9.41
16	0.28,200.0,9.41
17	0.3,200.0,9.41
18	0.32,200.0,9.41
19	0.34,200.0,9.41
20	0.36,200.0,9.41
21	0.38,200.0,9.41
22	0.4,200.0,9.41
23	0.42,200.0,9.41
24	0.44,200.0,9.41
25	0.46,200.0,9.41

Figura 7: Datos experimentales obtenidos mediante encoder.

Durante la implementación se observaron diferencias entre simulación y experimento debido a factores no considerados en el modelo matemático:

- Fricción mecánica.
- Saturación del PWM.
- Variaciones de carga.
- Ruido de medición.

11. Análisis de resultados

El modelo obtenido permitió representar correctamente la dinámica principal del actuador, obteniendo un ajuste superior al 80 % durante la identificación.

El controlador discreto diseñado mediante LGR permitió realizar el seguimiento de la referencia de velocidad con una respuesta estable y rápida.

Las diferencias observadas entre simulación y pruebas físicas son esperadas debido a que el modelo identificado no contempla completamente los efectos mecánicos presentes en el sistema real.

12. Conclusiones

- La identificación experimental permitió obtener un modelo simplificado del actuador adecuado para el diseño del controlador.
- El controlador discreto mediante LGR permitió realizar una implementación eficiente sobre una plataforma embebida ESP32.
- La selección del periodo de muestreo permitió garantizar una ejecución adecuada del algoritmo de control en tiempo real.
- Las pruebas físicas validaron el funcionamiento del sistema y permitieron observar la influencia de fenómenos reales como fricción y ruido.
- La metodología aplicada permite extender el diseño hacia otros actuadores presentes en sistemas robóticos.